

[Name der Hochschule]

[Art der Arbeit]

[Titel der Fallstudie zum Technologie-Template]

[Untertitel]

[Aufgabenstellung]

Kurs: [Kurs / Modul]
Studiengang: [Studiengang]
Autor: [Vorname Nachname]
Matrikelnummer: [Matrikelnummer]
Tutor: [Tutor / Betreuer]
Abgabedatum: [Monat Jahr]

Inhaltsverzeichnis

Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Abkürzungsverzeichnis	VI
1 Einleitung	1
1.1 Ausgangslage	1
1.2 Problemstellung	1
1.3 Zielsetzung	1
1.4 Fragestellungen	2
1.5 Methodisches Vorgehen	2
1.6 Aufbau der Fallstudie	3
2 Anforderungen an das Technologie-Template	4
2.1 Zielbild des Technologie-Templates	4
2.2 Anforderungen an Wiederverwendbarkeit und Standardisierung	4
2.3 Anforderungen für Web-, Cloud- und Desktop-Anwendungen	5
2.4 Qualitäts- und Wartbarkeitsanforderungen	5
2.5 Abgrenzung	5
3 Architektur des Technologie-Templates	6
3.1 Gesamtarchitektur	6
3.2 Frontend mit Vite und TypeScript	8
3.3 Backend mit FastAPI	8
3.4 Desktop-Integration mit Tauri und Rust	9
3.5 Shared Contracts und Schnittstellen	10
3.6 Konfigurationsmodell	10
3.7 Persistenz mit PostgreSQL, SQLAlchemy und Alembic	11
4 Projektprofile und Feature-Modell	12
4.1 Grundprinzip des Profilmodells	12
4.2 Profil Web-only	14
4.3 Profil Web-cloud	14

4.4	Profil Desktop-local	15
4.5	Profil Desktop-cloud	15
4.6	Profil Full-platform	15
4.7	Optionale Capabilities	15
4.8	Single-Repository-Strategie	16
5	Tooling und Entwicklungsworkflow	17
5.1	Rolle von control.py	17
5.2	Projektinitialisierung und Generierung	18
5.3	Systemprüfung und Installation	18
5.4	Entwicklungsbetrieb	18
5.5	Tests und Builds	19
5.6	Release- und Packaging-Workflow	19
5.7	Entwickler-Onboarding	19
6	Qualitätssicherung und Verifikation	19
6.1	Teststrategie	19
6.2	Continuous Integration	20
6.3	Abnahme und technische Verifikation	20
6.4	Reproduzierbarkeit	20
6.5	Security und Konfigurationsgrenzen	20
6.6	Extern verbleibende Prüfungen	20
7	Bewertung des Technologie-Templates	20
7.1	Produktivitätswirkung	20
7.2	Stärken	20
7.3	Schwächen und Grenzen	21
7.4	Wartungsaufwand	21
7.5	Vergleich mit alternativen Template-Strategien	21
7.6	Gesamtbewertung	21
8	Einsatzmöglichkeiten und Transfer auf Kundenprojekte	21
8.1	Geeignete Projekttypen	21
8.2	Ungeeignete und eingeschränkt geeignete Projekttypen	21
8.3	Analyse von Kundenanforderungen	21

8.4	Auswahl des Projektprofils	21
8.5	Generierung eines Kundenprojekts	21
8.6	Überführung in die Produktentwicklung	22
9	Schlussbetrachtung	22
9.1	Zusammenfassung der Ergebnisse	22
9.2	Beantwortung der Fragestellungen	22
9.3	Kritische Würdigung	22
9.4	Ausblick	22
9.5	Fazit	22
	Literaturverzeichnis	23

Abbildungsverzeichnis

1	Methodische Schritte der Fallstudie	3
2	Gesamtarchitektur des Technologie-Templates	6
3	Komponenten und Verantwortungsgrenzen im Master Repository	7
4	Profilabhängige Laufzeitarchitektur	7
5	Providerneutrale Deployment-Architektur	9
6	Konfigurationspriorität und Sicherheitsgrenzen	11
7	Ableitung eines Produktprojekts aus einer gemeinsamen Template-Basis	12
8	Entscheidungsfluss für Profil und anschließende Capability-Auswahl	13
9	Strukturelle Feature-Zuordnung der fünf Profil-Presets	14
10	Implementierte Abhängigkeiten des Feature-Katalogs	16
11	Befehlsgruppen des zentralen Projekt-Toolings	17
12	Workflow der profilgesteuerten Projektgenerierung	18
13	Grundstruktur des Entwicklungsworkflows	18
14	Plattformübergreifendes Modell für Desktop-Releases	19
15	Nachweiskette der technischen Verifikation	19
16	Struktur der profilbezogenen Verifikationsmatrix	20
17	Offene Bewertungsstruktur für die Projekteignung	21
18	Prozess vom Kundenbedarf bis zur Auslieferung	21

Tabellenverzeichnis

1	Zentrale Anforderungen an das Technologie-Template	4
2	Anforderungsmatrix der unterstützten Projektarten	5
3	Deklarierte und aufgelöster Technologie-Stack im untersuchten Commit	8
4	Deklarierte Basisfeatures und resultierende Laufzeitverzeichnisse	14
5	Struktur zur Dokumentation der technischen Verifikation	20
6	Struktur zur Gegenüberstellung von Stärken und Grenzen	20
7	Struktur zur Bewertung der Projekteignung	21

Abkürzungsverzeichnis

Abk. Bedeutung

1 Einleitung

1.1 Ausgangslage

Die Einrichtung neuer Softwareprojekte erfordert wiederkehrende Entscheidungen zu Architektur, Technologien, Schnittstellen, Konfiguration sowie Test-, Build- und Deployment-Prozessen. Werden diese Grundlagen für jedes Vorhaben unabhängig erstellt, entstehen unterschiedliche technische Ausgangslagen. Web-, Cloud- und Desktop-Anwendungen stellen zudem verschiedene Anforderungen an Komponenten und Bereitstellung. Bei plattformübergreifenden Projekten müssen gemeinsame Bestandteile deshalb klar von spezifischen Erweiterungen getrennt werden.

Eine standardisierte technische Basis kann wiederkehrende Entscheidungen bündeln und ein konsistentes Entwicklungsmodell bereitstellen. Gegenstand der Fallstudie ist das Projekt „Template-Projekte“, ein wiederverwendbares Full-Stack-Technologie-Template für Web-, Cloud- und Desktop-Projekte. Seine Ausgestaltung und Eignung werden im weiteren Verlauf systematisch untersucht (kleiveist, 2026g).

1.2 Problemstellung

Die wiederholte Initialisierung technisch ähnlicher Projekte erzeugt Aufwand außerhalb der produktspezifischen Entwicklung. Ohne gemeinsame Baseline können sich Projektstruktur, Werkzeugauswahl, Konfiguration und Build-Prozesse trotz vergleichbarer Anforderungen unterscheiden. Getrennt gepflegte Ausgangsstände für Web-, Cloud- und Desktop-Anwendungen erhöhen zudem den Wartungsaufwand und begünstigen auseinanderlaufende Technologie- und Dokumentationsstände.

Die zentrale Herausforderung besteht darin, Einheitlichkeit und Flexibilität zu verbinden. Das Template muss genügend gemeinsame Struktur für Standardisierung und Wiederverwendung vorgeben, darf ein Projekt jedoch nicht mit unnötigen Komponenten belasten. Untersucht wird daher, wie eine gemeinsame Basis, Projektprofile und optionale Erweiterungen diesen Zielkonflikt adressieren und wo Anpassungsbedarf verbleibt (kleiveist, 2026c, 2026d).

1.3 Zielsetzung

Ziel der Fallstudie ist die strukturierte Untersuchung des Projekts „Template-Projekte“. Analysiert werden die Architektur, die Aufteilung in gemeinsame und profilspezifische Bestandteile sowie die unterstützenden Entwicklungs- und Qualitätssicherungsprozesse (kleiveist, 2026c, 2026h).

Auf dieser Grundlage werden Wiederverwendbarkeit, Standardisierung und das Produktivitätspotenzial des Templates sowie seine Eignung für unterschiedliche Projektarten bewertet. Die Untersuchung stützt sich auf vorhandene Projektartefakte und Verifikationsnachweise. Daraus sollen Einsatzfelder und Auswahlkriterien für zukünftige Kundenprojekte sowie technische Grenzen und externe Voraussetzungen abgeleitet werden (kleiveist, 2026f). Eine universelle Eignung für sämtliche Softwareklassen oder eine produktspezifische Facharchitektur wird nicht beansprucht.

1.4 Fragestellungen

Aus der beschriebenen Problemstellung und Zielsetzung ergeben sich sechs zentrale Untersuchungsfragen. Sie strukturieren die Analyse des Templates und bilden den Bezugsrahmen für seine spätere Bewertung:

1. Welche fachlich-technischen, qualitativen und betrieblichen Anforderungen werden durch das Technologie-Template adressiert?
2. Wie unterstützen die Architektur und das Profilmodell die Bereitstellung unterschiedlicher Varianten für Web-, Cloud- und Desktop-Projekte?
3. Inwiefern fördern der gemeinsame technische Kern und die Single-Repository-Strategie die Wiederverwendung und Standardisierung zwischen Projekten?
4. Welches Potenzial bietet das Template, wiederkehrende Aufwände bei Projektinitialisierung, Entwicklungsworkflow, Qualitätssicherung und Bereitstellung zu reduzieren?
5. Für welche Projektarten und Kundenanforderungen stellt das Template eine geeignete technische Ausgangsbasis dar?
6. Welche technischen Grenzen, Wartungsaufwände, Risiken und externen Abhängigkeiten sind bei seinem Einsatz zu berücksichtigen?

Die Fragen sind miteinander verknüpft: Aussagen zu Produktivität und Eignung setzen zunächst eine nachvollziehbare Betrachtung der Anforderungen, der Architektur und der vorhandenen Verifikation voraus. Ihre Beantwortung wird daher nicht isoliert, sondern entlang der aufeinander aufbauenden Kapitel der Fallstudie vorgenommen.

1.5 Methodisches Vorgehen

Die Untersuchung ist als technische Fallstudie des implementierten Templates angelegt (Runeson & Höst, 2009). Ausgangspunkt ist eine Bestandsaufnahme des Master Repository. Dabei werden die Repository-Struktur, die enthaltenen Komponenten und die dokumentierten Verantwortungsgrenzen erfasst. Anschließend werden die Architekturentscheidungen und die Beziehungen zwischen Frontend, Backend, Desktop-Integration, gemeinsam genutzten Schnittstellen, Persistenz und Deployment betrachtet. Die Analyse beschreibt den vorhandenen Gegenstand und trennt belegbare Projekteigenschaften von Annahmen oder noch offenen Prüfpunkten (kleiveist, 2026c).

Darauf aufbauend wird das Profilmodell untersucht. Im Mittelpunkt stehen die Abgrenzung der Profile, die Aufteilung zwischen gemeinsamem Kern und profilspezifischen Bestandteilen sowie das Modell optionaler Capabilities. Die Betrachtung des Python-basierten Toolings ergänzt diese strukturelle Analyse um den praktischen Entwicklungsablauf. Hierzu werden insbesondere Projektgenerierung, Systemprüfung, Installation, Entwicklungsbetrieb, Tests, Builds und Release-Vorbereitung als zusammenhängender Workflow betrachtet. Die maSSgeblichen Profil- und Werkzeugbeschreibungen werden dabei als projektinterne Primärquellen herangezogen (kleiveist, 2026d, 2026h).

Für die Verifikation werden vorhandene automatisierte Tests, Build-Ergebnisse und dokumentierte Abnahmen den jeweils untersuchten Anforderungen und Architekturentscheidungen zugeordnet. Externe oder produktspezifische Prüfschritte werden davon abgegrenzt. Auf dieser Evidenzbasis erfolgt die Bewertung von Wiederverwendbarkeit, Standardisierung, Produktivitätspotenzial, Stärken und Grenzen. Abschließend werden daraus Kriterien für geeignete Einsatzfelder und für die Übertragung auf zukünftige Kundenprojekte abgeleitet (kleiveist, 2026f).

Abbildung 1 fasst diese aufeinander aufbauenden Untersuchungsschritte zusammen. Die vertikale Darstellung verdeutlicht, dass die Bewertung und der Transfer erst auf Grundlage der zuvor erhobenen und analysierten Projektinformationen erfolgen.

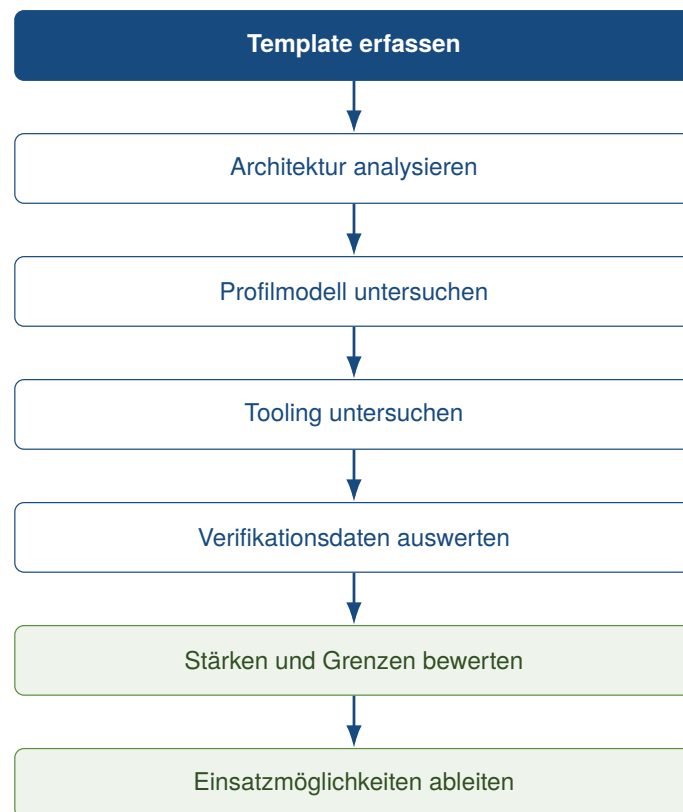


Abbildung 1: Methodische Schritte der Fallstudie

Quelle: Eigene Darstellung.

1.6 Aufbau der Fallstudie

Nach der Einleitung konkretisiert Abschnitt 2 die Anforderungen und die Abgrenzung des Technologie-Templates. Darauf folgt in Abschnitt 3 die Untersuchung seiner Gesamtarchitektur und der beteiligten technischen Komponenten. Abschnitt 4 betrachtet anschließend das Profil- und Feature-Modell einschließlich der optionalen Capabilities und der Single-Repository-Strategie. Das zentrale Projekt-Tooling sowie der Entwicklungs-, Build- und Release-Workflow werden in Abschnitt 5 untersucht.

Abschnitt 6 behandelt die Qualitätssicherung, die technische Verifikation und extern verbleibende Prüfungen. Auf dieser Grundlage bewertet Abschnitt 7 das Template hinsichtlich Produktivitätswirkung, Stärken, Grenzen und Wartungsaufwand. Abschnitt 8 überträgt die gewonnenen Erkenntnisse auf die Auswahl und Generierung zukünftiger Kundenprojekte. Die Arbeit endet mit der Schlussbetrachtung

in Abschnitt 9, in der die Ergebnisse gebündelt, die Fragestellungen beantwortet und offene Entwicklungsperspektiven eingeordnet werden.

2 Anforderungen an das Technologie-Template

2.1 Zielbild des Technologie-Templates

Das Template soll eine wiederverwendbare technische Ausgangsbasis für Web-, Cloud- und Desktop-Projekte bilden. Wiederverwendung setzt dabei geplante Gemeinsamkeiten und beherrschte Variabilität voraus (Clements & Northrop, 2001; Krueger, 1992). Projektstruktur, Entwicklungsworkflow, Tooling, Dokumentationsprinzipien, Konfigurationslogik, Tests und Schnittstellen sollen deshalb in einem gemeinsamen Kern zentral gepflegt werden.

Nicht jedes Projekt benötigt sämtliche Komponenten. Profile sollen passende Kombinationen des Kerns auswählen; optionale Capabilities wie PostgreSQL sollen kompatible Backend-Profile unabhängig vom Plattformprofil erweitern (kleiveist, 2026d). Bereits im Zielbild sind Frontend, Backend, Desktop-Schicht, Persistenz, Tooling und Deployment als getrennte Verantwortungsbereiche vorgesehen. Ob die Umsetzung diese Anforderungen erfüllt, wird erst in den nachfolgenden Architektur-, Verifikations- und Bewertungskapiteln geprüft.

2.2 Anforderungen an Wiederverwendbarkeit und Standardisierung

Einheitlich bedeutet nicht identisch: Die Baseline soll gemeinsame Strukturen und öffentliche Arbeitsabläufe vorgeben, zugleich aber kontrollierte Profilvarianten zulassen. Unabhängige Kopien sind zu vermeiden, weil sie Änderungen des gemeinsamen Fundaments mehrfach erforderlich machen und langfristig auseinanderlaufen können. Tabelle 1 fasst die daraus abgeleiteten, später überprüfbaren Anforderungen zusammen.

Tabelle 1: Zentrale Anforderungen an das Technologie-Template

ID	Anforderung	Operationalisiertes Ziel	spätere Prüfbasis
A-01	Baseline	Grundstruktur muss generierbar sein.	Projektgenerierung
A-02	Struktur	Verantwortlichkeiten liegen an konsistenten Stellen.	Repository-Struktur
A-03	Profile	Nur vorgesehene Komponenten werden aktiviert.	Profilmodell
A-04	Workflows	Prüfen, Initialisieren, Installieren, Starten, Testen und Bauen besitzen gemeinsame Einstiegspunkte.	Tooling
A-05	Wartbarkeit	Gemeinsame Bestandteile werden zentral gepflegt.	Repository-Strategie
A-06	Nachvollziehbarkeit	Änderungen und Konfiguration sind versioniert und dokumentiert.	Versionsstand, Dokumentation
A-07	Erweiterbarkeit	Optionale Fähigkeiten besitzen dokumentierte Abhängigkeiten.	Capability-Modell

Quelle: Eigene Darstellung.

Die Tabelle definiert Soll-Kriterien, keine festgestellten Eigenschaften. Ihre Erfüllung wird anhand der genannten Prüfbasis in den späteren Kapiteln beurteilt.

2.3 Anforderungen für Web-, Cloud- und Desktop-Anwendungen

Alle Varianten sollen eine gemeinsame Frontend-Basis, Projektstruktur, Konfigurationslogik sowie ein einheitliches Test- und Dokumentationsmodell nutzen. Browserprojekte benötigen keine native Desktop-Schicht und nur bei Bedarf ein Backend. Cloudfähige Varianten erfordern eine serverseitige API, konfigurierbare Deployment-Grenzen sowie Health- und Readiness-Prüfbarkeit. Desktopvarianten sollen dasselbe Frontend in einer nativen Laufzeit verwenden und Weblogik von plattformspezifischer Integration trennen. Kombinierte Varianten verbinden Desktop-Client, Backend und gegebenenfalls Browserzugriff.

Tabelle 2: Anforderungsmatrix der unterstützten Projektarten

Anforderung	Web	Cloud	Desktop lokal	Desktop + Cloud
Frontend	erforderlich	erforderlich	erforderlich	erforderlich
Backend / API	optional	erforderlich	nicht grundsätzlich	erforderlich
Desktop-Laufzeit	nein	nein	erforderlich	erforderlich
Deployment-Grenze	optional	erforderlich	nein	erforderlich
Persistenz	optional	optional	produktspezifisch	optional

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026d).

Die Matrix formuliert Anforderungen, keine Prüfergebnisse. PostgreSQL ist dabei ausschliesslich eine optionale Capability für backendfähige Profile und kein eigenes Plattformprofil (kleiveist, 2026d). Öffentliche Client-Konfiguration und serverseitige Werte einschliesslich Geheimnissen sind voneinander zu trennen; eine bestimmte Cloud-Plattform wird nicht vorausgesetzt.

2.4 Qualitäts- und Wartbarkeitsanforderungen

Die Qualitätsanforderungen orientieren sich an überprüfbaren Merkmalen des Produktqualitätsmodells nach ISO/IEC 25010 (ISO/IEC, 2023). Komponenten und Profile sollen klare Verantwortlichkeiten besitzen, getrennt testbar und durch dokumentierte Architektur-, Profil- und Workflowentscheidungen wartbar sein. Neue optionale Fähigkeiten sowie Technologieaktualisierungen sollen ohne Duplizierung der gesamten Baseline möglich bleiben; daraus folgt keine Annahme, dass Aktualisierungen automatisch risikofrei sind.

Gleiche Eingaben und Konfigurationen sollen nachvollziehbare Generierungs- und Build-Ergebnisse ermöglichen. Reproduzierbare Builds erhöhen die Prüfbarkeit der Beziehung zwischen Quellstand und Artefakt (Lamb & Zacchioli, 2022). Ferner sollen relevante Zielartefakte kontrolliert erzeugbar, Konfigurationsquellen und Prioritäten verständlich definiert und serverseitige Geheimnisse von Frontend- und Client-Konfiguration getrennt sein. Die tatsächliche Erfüllung wird in Kapitel 6 untersucht.

2.5 Abgrenzung

Das Zielbild ist keine universelle Softwareplattform. Im Mittelpunkt stehen wiederkehrende Web-, Backend-, Desktop- und Full-Stack-Business-Anwendungen. Geschäftsregeln, Domänen- und Kundendatenmodelle sowie produktspezifische Authentifizierungs-, Rollen- und Berechtigungskonzepte gehören nicht zur technischen Baseline. Ebenso wird kein bestimmter Cloud-Provider vorausgesetzt (kleiveist, 2026c).

Stark plattformspezifische native Anwendungen, Spiele und hochspezialisierte Echtzeitsysteme zählen nicht automatisch zum Zielbereich. Dieses Kapitel definiert lediglich Anforderungen und Grenzen; eine Eignungszusage erfolgt erst auf Grundlage der späteren Verifikation und Bewertung.

3 Architektur des Technologie-Templates

3.1 Gesamtarchitektur

Die untersuchte Baseline ist ein Master-Repository mit gemeinsam gepflegtem Kern und deklarativen Varianten, kein zur Laufzeit zusammenhängender Monolith. Profile wählen wiederverwendbare Features aus; optionale Capabilities ergänzen nur compatible Profile. Dieses Vorgehen überträgt das Prinzip beherrschter Variabilität aus Software-Produktlinien auf die Projektstruktur und adressiert damit die in Kapitel 2 geforderte zentrale Wartbarkeit ohne getrennte Template-Kopien (Clements & Northrop, 2001; kleiveist, 2026d).

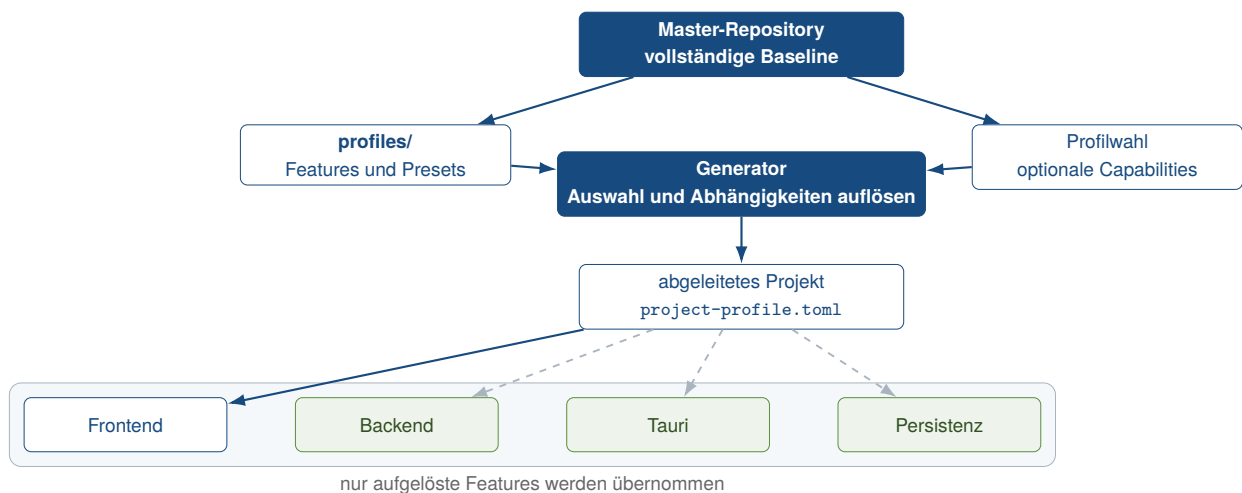


Abbildung 2: Gesamtarchitektur des Technologie-Templates

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026c) und kleiveist (2026d).

Abbildung 2 trennt deshalb die vollständige Baseline von einem abgeleiteten Projekt. Der Generator löst Profil und Capabilities auf, übernimmt nur die zugehörigen Pfade und hält das Ergebnis in `project-profile.toml` fest. Frontend, Backend, Tauri und Persistenz bleiben eigenständige, teilweise optionale Bausteine (kleiveist, 2026c).

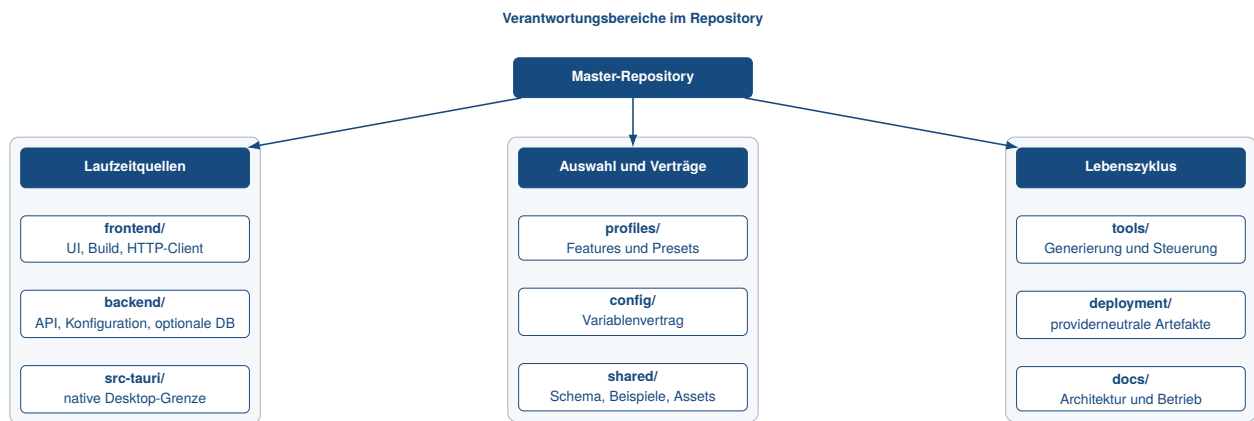


Abbildung 3: Komponenten und Verantwortungsgrenzen im Master Repository

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026c).

Abbildung 3 ordnet die Verantwortungen konkreten Verzeichnissen zu. `frontend/` enthält Darstellung und HTTP-Client, `backend/` die serverseitige API, `src-tauri/` die native Grenze. `shared/` hält frameworkneutrale Artefakte; `profiles/` und `config/` beschreiben Struktur beziehungsweise Laufzeitwerte. Tooling und Deployment steuern diese Komponenten, gehören aber nicht zu deren Produktlaufzeit.

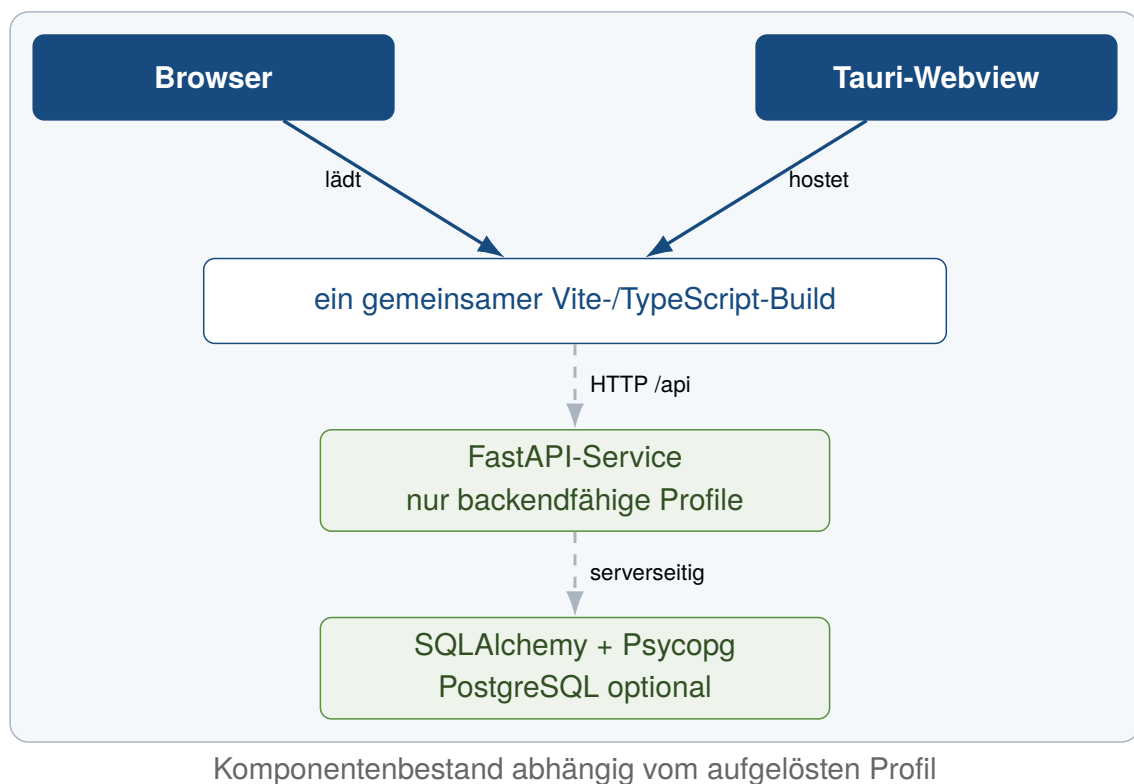


Abbildung 4: Profilabhängige Laufzeitarchitektur

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026c) und kleiveist (2026d).

Zur Laufzeit wird derselbe Vite-Build entweder vom Browser oder in einer Tauri-Webview geladen (Abbildung 4). Nur backendfähige Profile ergänzen die Kommunikation über die HTTP-Grenze zu FastAPI; ausschliesslich das Backend darf eine aktivierte Persistenzschicht ansprechen. Damit bleiben Client, Server und Datenhaltung getrennt test- und erweiterbar, ohne dass jedes Profil alle Laufzeiteinheiten

enthalten muss.

3.2 Frontend mit Vite und TypeScript

Das Frontend bildet die gemeinsame Präsentationsschicht aller fünf Profile. Vite stellt im Entwicklungsbetrieb den Modulserver bereit und erzeugt mit `vite build` statische Artefakte; der vorgeschaltete TypeScript-Lauf prüft den strikt konfigurierten Quellcode ohne JavaScript-Ausgabe. Vitest deckt den HTTP-Client ab. Diese Aufgabenteilung entspricht den dokumentierten Rollen von Vite als Entwicklungs- und Build-Werkzeug sowie TypeScript als statischer Typprüfung (Microsoft, 2026; VoidZero Inc. and Vite Contributors, 2026).

`frontend/src/main.ts` liest das generierte Profilmodul und zeigt die Backend-Integration nur bei aktiviertem Feature. Alle Aufrufe liegen in `frontend/src/api/`; `backend.ts` verlangt dann eine öffentliche `VITE_API_BASE_URL` und ruft `/api/health` auf. Dieselben Quellen werden im Browser und in Tauri verwendet. Serverlogik, Geheimnisse und direkter Datenbankzugriff bleiben außerhalb dieser Grenze; Web-only und Desktop-local benötigen folglich kein Backend (kleiveist, 2026c, 2026d).

Die im untersuchten Commit deklarierten Bereiche und aufgelösten Versionen sind in Tabelle 3 zusammengefasst.

Tabelle 3: Deklarierter und aufgelöster Technologie-Stack im untersuchten Commit

Technologie	Rolle im Template	Version / Quelle	Projektbeleg
Vite	Entwicklungsserver und Frontend-Build	Bereich ab 6.0.7; Lock: 6.4.2	<code>frontend/package.json</code> , <code>frontend/package-lock.json</code>
TypeScript	statische Prüfung des Frontends	Bereich ab 5.7.3; Lock: 5.9.3	<code>frontend/package.json</code> , <code>frontend/tsconfig.json</code>
FastAPI	HTTP-API und Validierung	≥ 0.111 , < 1 ; Produktion: 0.111.0	<code>backend/requirements.txt</code> , <code>backend/requirements-production.txt</code>
Tauri	Desktop-Webview und Packaging-Grenze	Major 2; Lock: 2.11.5	<code>src-tauri/Cargo.toml</code> , <code>src-tauri/Cargo.lock</code>
Rust	native Kompositionsschicht	mindestens 1.77; Edition 2021	<code>src-tauri/Cargo.toml</code>
PostgreSQL	optionale relationale Laufzeiteinheit	Container: 16.4-alpine3.20	<code>deployment/compose.yaml</code>
SQLAlchemy	Engine-, Session- und Modellbasis	≥ 2 , < 3 ; Produktion: 2.0.36	<code>backend/requirements-database*.txt</code>
Alembic	explizite Schemamigrationen	≥ 1.13 , < 2 ; Produktion: 1.14.0	<code>backend/requirements-database*.txt</code>
Psycopg	PostgreSQL-Treiber	≥ 3.2 , < 4 ; Produktion: 3.2.3	<code>backend/requirements-postgres*.txt</code>

Quelle: Eigene Darstellung auf Grundlage des geprüften Projektstands (kleiveist, 2026g).

3.3 Backend mit FastAPI

Das Backend ist eine separate HTTP-Schicht der Profile Web-cloud, Desktop-cloud und Full-platform. `backend/app/main.py` erzeugt die FastAPI-Anwendung, lädt typisierte Einstellungen, richtet CORS ein und bindet den Router unter `/api` ein. Der derzeitige Router enthält ausschließlich `/health` und `/ready`; produktspezifische Dienste, Fachmodelle und Authentifizierung sind bewusst nicht vorgegeben. FastAPI stellt hierfür die HTTP-Routen, Validierung und OpenAPI-Beschreibung bereit (kleiveist, 2026c; Ramírez, 2026).

Liveness meldet nur den Prozesszustand. Readiness prüft bei aktivierter Datenbankfähigkeit zusätz-

lich mit `SELECT 1` und antwortet bei fehlender Konfiguration oder Verbindung mit HTTP 503, ohne interne Fehler offenzulegen. Tests adressieren Router, Einstellungen und diese Abhängigkeit getrennt. Die Architektur hält Handler klein; erst ein abgeleitetes Produkt ergänzt Service- oder Domänenmodule.

Auch im Deployment bleiben die Einheiten getrennt (Abbildung 5). Der Vite-Build kann statisch bereitgestellt werden, während FastAPI in einem eigenen, nicht privilegierten Container läuft. Compose simuliert diese providerneutrale Grenze lokal; PostgreSQL wird nur über ein explizites Profil ergänzt (kleiveist, 2026b).

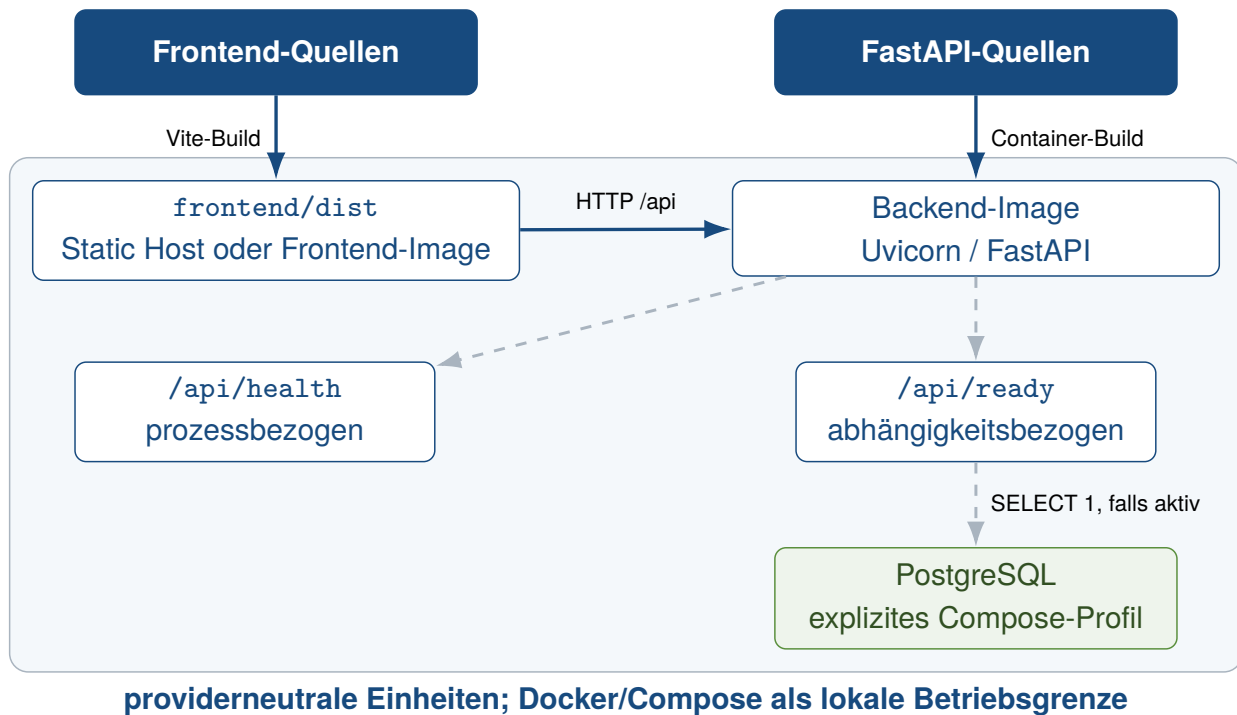


Abbildung 5: Providerneutrale Deployment-Architektur

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026b).

3.4 Desktop-Integration mit Tauri und Rust

Tauri bildet eine eigene native Grenze, ohne das Frontend zu duplizieren. `src-tauri/tauri.conf.json` startet für die Entwicklung ausschliesslich den Vite-Server und verwendet für Builds `frontend/dist`. Der Rust-Einstieg erstellt lediglich die Tauri-Anwendung; native Fachkommandos oder ein eingebetteter FastAPI-Sidecar sind im Template nicht implementiert. Dies nutzt Tauris Fähigkeit, Web-Frontends in einer nativen Webview zu hosten, während Rust die native Kompositionsgrenze bereitstellt (Klabnik et al., 2026; kleiveist, 2026c; Tauri Contributors, 2026b).

Die Standard-Capability gewährt nur `core:default`; eine Content Security Policy begrenzt lokale Ressourcen und die dokumentierten Entwicklungsendpunkte. Weitere native Berechtigungen müssen produktspezifisch und möglichst eng ergänzt werden (Tauri Contributors, 2026a). Im Profil Desktop-local bleibt Funktionalität lokal im Frontend oder in später ergänzten Tauri-Kommandos. Desktop-cloud und Full-platform verwenden dagegen die öffentliche FastAPI-URL. Packaging, Signierung und die Entscheidung zwischen Remote-API, Sidecar oder lokalen Kommandos sind Produktentscheidungen und nicht Teil dieser Baseline.

3.5 Shared Contracts und Schnittstellen

`shared/` gehört zum Kern jedes generierten Projekts und enthält statische Assets, ein JSON-Schema nach Draft 2020-12 sowie je ein gültiges und ungültiges Beispiel. Die Schema-Tests des Toolings validieren diese Beispiele; die Ablage bleibt damit serialisierbar und frameworkneutral. Sie definiert jedoch noch keine produktbezogene API und enthält keinen ausführbaren Frameworkcode (kleiveist, 2026c, 2026g).

Der Implementierungsabgleich begrenzt diese Aussage: Weder Frontend noch Backend importieren das vorhandene Eingabeschema zur Laufzeit. Auch der Health-Vertrag ist nicht daraus generiert, sondern als TypeScript-Interface und FastAPI-Antwort manuell gespiegelt. Eine automatische Codegenerierung oder Synchronisierung ist nicht vorhanden. Vertragsänderungen erfordern daher derzeit eine bewusste Anpassung beider Seiten und ihrer Konsumententests. Diese Abweichung zwischen der dokumentierten Zielrolle gemeinsam konsumierter Verträge und der aktuellen Nutzung ist eine offene Erweiterungsgrenze, kein Nachweis einer bereits automatisierten Schnittstellenkopplung.

3.6 Konfigurationsmodell

Projektstruktur und Laufzeitkonfiguration sind getrennte Achsen: `project-profile.toml` bestimmt die vorhandenen Features, `config/environment.toml` den Variablenvertrag. Der zentrale Resolver berücksichtigt nur Variablen der aktiven Features und wendet in absteigender Priorität CLI-Override, Prozessumgebung, lokale `.env` und Vertragsdefault an. Ableitungen wie die lokale API-URL bleiben als Quelle nachvollziehbar (kleiveist, 2026e).

Abbildung 6 zeigt die Adapter- und Sicherheitsgrenzen. Vite bettet ausschließlich die explizit freigegebene `VITE_API_BASE_URL` ein. FastAPI verarbeitet serverseitige Werte typisiert; `DATABASE_URL` bleibt ein maskiertes Secret. Tooling gibt nur relevante Werte an Kindprozesse weiter, und die Tauri-Umgebung entfernt bekannte Secret-Namen. Ungültige Hosts, Ports, Origins oder erforderliche fehlende Werte führen zu einem kontrollierten Abbruch, ohne den geheimen Eingabewert in der Fehlermeldung auszugeben.

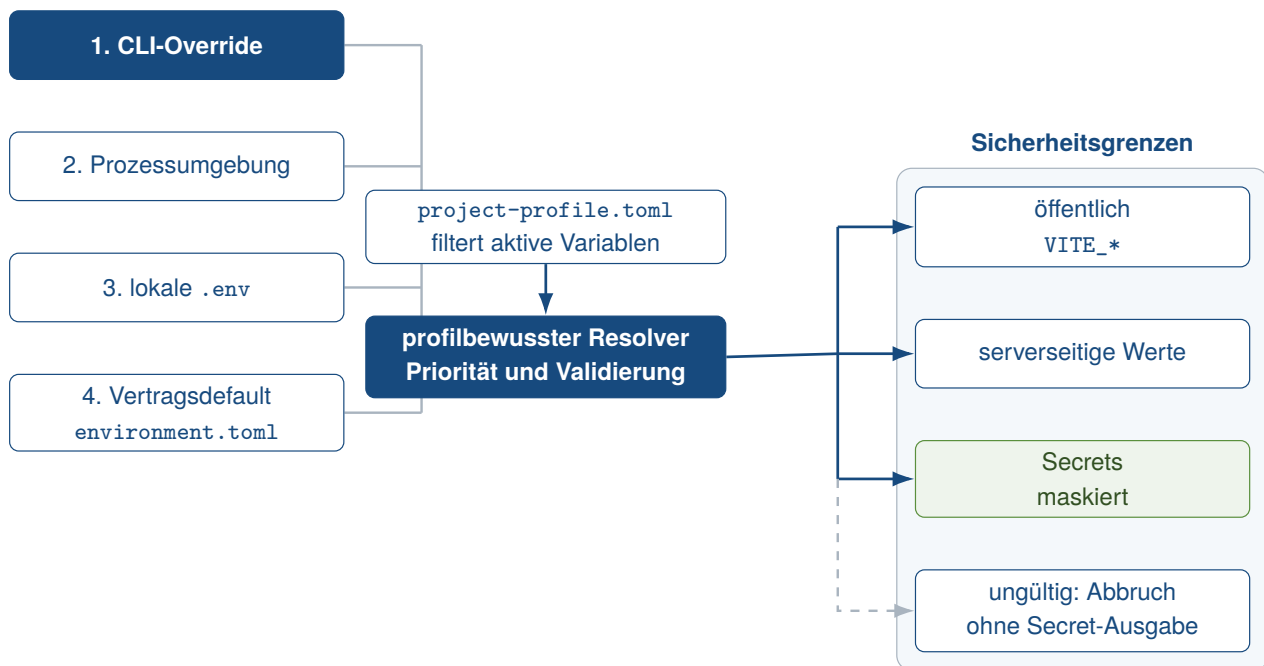


Abbildung 6: Konfigurationspriorität und Sicherheitsgrenzen

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026e).

3.7 Persistenz mit PostgreSQL, SQLAlchemy und Alembic

PostgreSQL ist keine Pflichtkomponente und kein eigenes Profil. Die wählbare Capability `postgres` setzt `database` voraus, das wiederum ein Backend benötigt; `Web-only` und `Desktop-local` werden deshalb abgewiesen. Die generische Datenbankfähigkeit enthält SQLAlchemy-Engine, Sessions und Alembic, während die PostgreSQL-Erweiterung den Psycopg-3-Treiber, URL-Vorgaben und Integrationstests ergänzt. PostgreSQL selbst bleibt eine getrennte Laufzeiteinheit (kleiveist, 2026a; PostgreSQL Global Development Group, 2026).

`backend/app/db/engine.py` erzeugt die Engine erst beim ersten Zugriff und aktiviert `pool_pre_ping`; Sessions werden über eine Generator-Dependency geschlossen. Die leere deklarative Base ist nur ein Erweiterungspunkt für spätere Fachmodelle. Das entspricht der von SQLAlchemy dokumentierten Trennung von Engine, Pool, Dialekt und Session sowie dem verzögerten Aufbau der ersten Verbindung (SQLAlchemy Authors and Contributors, 2026).

Alembic verwendet dieselbe Metadatenbasis für Online- und Offline-Migrationen. Da das Template keine Fachmodelle enthält, existiert noch keine initiale Revision; weder `create_all()` noch automatische Migrationen beim FastAPI-Start sind implementiert. Schemaänderungen erfolgen ausschließlich über explizite Alembic-Kommandos, deren Umgebung und Revisionsablauf die offizielle Dokumentation beschreibt (Bayer, 2026; kleiveist, 2026a). `/api/health` bleibt datenbankunabhängig, während `/api/ready` bei aktivierter Datenbankfähigkeit die Erreichbarkeit prüft.

4 Projektprofile und Feature-Modell

4.1 Grundprinzip des Profilmodells

Wie in Kapitel 3 beschrieben, bildet ein Master-Repository die gemeinsame technische Ausgangsbasis. Das Modell weist damit Merkmale featurebasierter Variabilitätsverwaltung und systematischer Softwarewiederverwendung auf, ohne allein deshalb eine vollständige Software-Produktlinie zu konstituieren (Clements & Northrop, 2001; Krueger, 1992). Der Katalog `profiles/features.toml` trennt vier Begriffe: Der *Core* umfasst die in jedes Produktprojekt kopierten Pfade `README.md`, `VERSION`, `.gitignore`, `config/`, `docs/`, `profiles/`, `shared/` und `tools/`. Ein *Feature* ordnet einer technischen Fähigkeit eigene Scaffold-Pfade und gegebenenfalls Abhängigkeiten zu. Ein *Profil* ist dagegen ein benanntes Preset zulässiger Features. Eine *optionale Capability* erweitert dieses Preset nach der Profilwahl; sie ist weder ein Plattformprofil noch automatisch Bestandteil eines Profils (kleiveist, 2026d, 2026g).

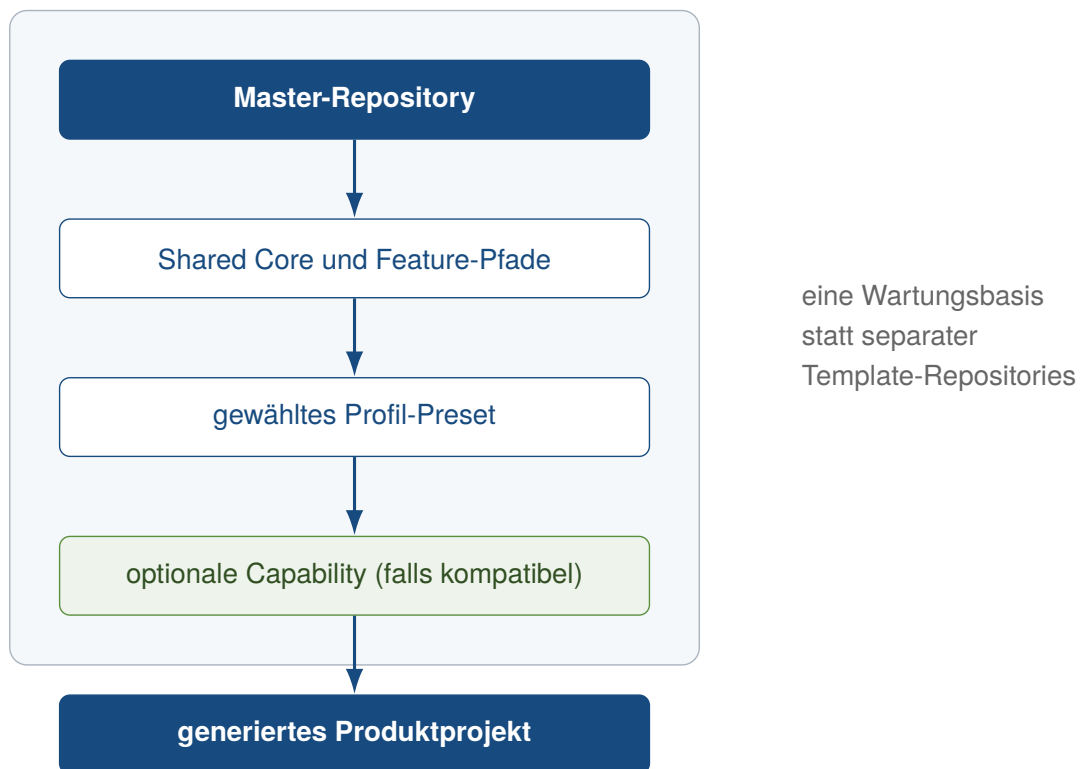


Abbildung 7: Ableitung eines Produktprojekts aus einer gemeinsamen Template-Basis

Abbildung 7 fasst diese Ebenen zusammen. Sie abstrahiert von den Laufzeitkomponenten und hebt stattdessen hervor, dass Core, Feature-Katalog und Presets in einer Wartungsbasis liegen. Der Generator löst das gewählte Preset und kompatible Capabilities auf, bildet aus Core- und Feature-Pfaden einen Scaffold-Plan und kopiert diesen in ein neues Produktprojekt. Nicht aktivierte Verzeichnisse werden somit überwiegend gar nicht ausgewählt; bei Webprofilen entfernt die Nachbearbeitung zusätzlich die Tauri-CLI aus den kopierten Frontend-Abhängigkeiten. Optional können Name, Slug und bei Tauri die Anwendungs-ID angepasst werden (kleiveist, 2026d, 2026h).

Die Auswahl in Abbildung 8 berücksichtigt, dass die gegenwärtigen Presets Backend und Cloud gemeinsam führen. Bei Desktopprojekten trennt der zusätzliche Browser-Produktkanal semantisch

full-platform von desktop-cloud; erst nach der Profilwahl wird eine Capability geprüft. Die anschließende Abhängigkeitsauflösung verhindert, dass dadurch stillschweigend ein fehlendes Plattformfeature ergänzt wird.

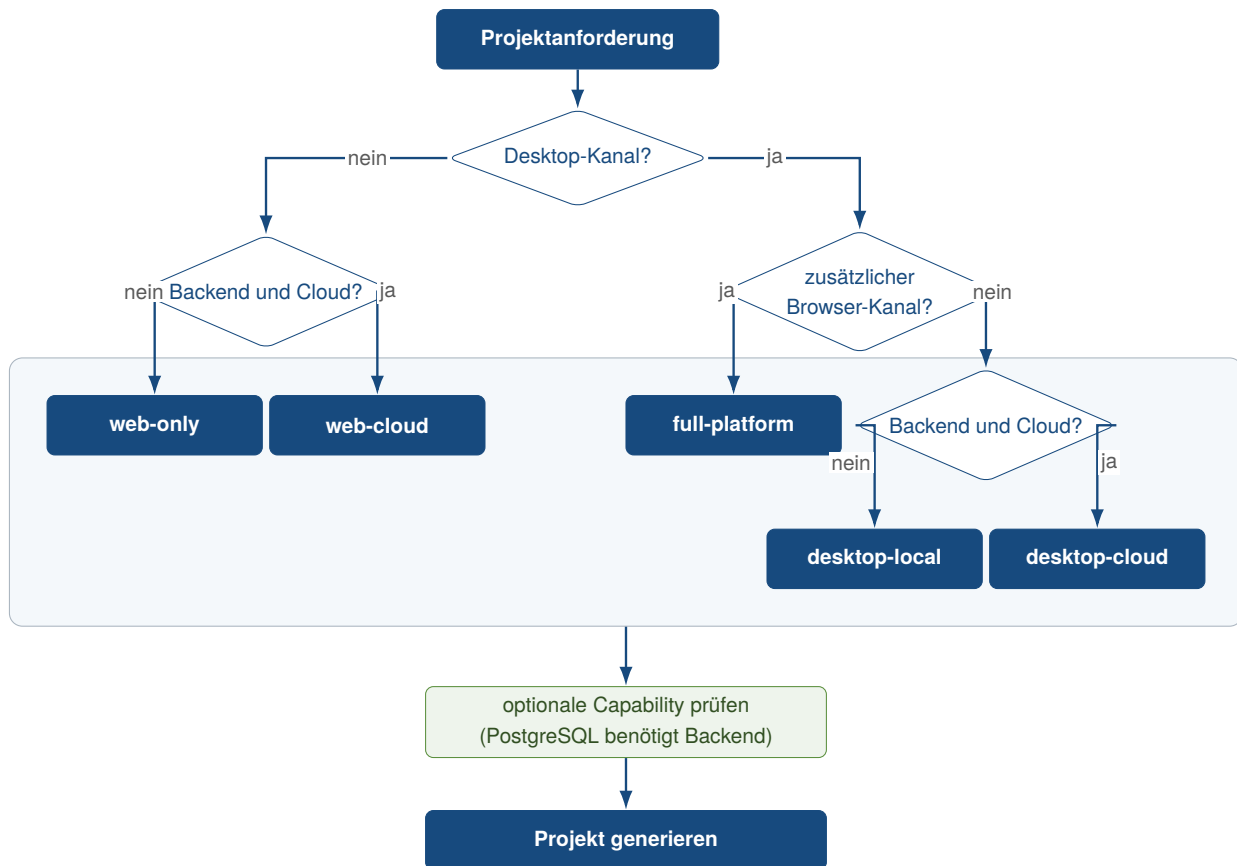


Abbildung 8: Entscheidungsfluss für Profil und anschließende Capability-Auswahl

Das aktive Manifest heit `project-profile.toml`. Es wird fr jedes generierte Projekt erzeugt. `schema_version` hlt die Schema-Version fest. `id`, `name` und `description` beschreiben das Profil. `optional_features` erfasst die angeforderten Erweiterungen; `features` enthlt die vollstndige Auflsung. Das Tooling ldt und validiert das Manifest. Dadurch behandelt es nur aktive Dienste und Workflows. Im Master nennt das Manifest `full-platform` als Profil. Zustzlich aktiviert es `postgres`; das Preset selbst bleibt davon unberhrt. Profilstruktur und Laufzeitwerte sind getrennte Achsen. Development-, Test- und Produktionswerte, Ports, URLs und Geheimnisse stammen aus dem in Abschnitt 3.6 beschriebenen Konfigurationsmodell (kleiveist, 2026e, 2026g).

Abbildung 9 ermglicht den schnellen strukturellen Vergleich. Tabelle 4 ergnzt dazu die exakt deklarierten Feature-IDs und die daraus ausgewhlten Laufzeitverzeichnisse. Insbesondere ist PostgreSQL bei keinem der fnf Presets voreingestellt.

Profil	Frontend	FastAPI	Tauri	Cloud	PostgreSQL
web-only	enthalten	–	–	–	inkompatibel
web-cloud	enthalten	enthalten	–	enthalten	optional
desktop-local	enthalten	–	enthalten	–	inkompatibel
desktop-cloud	enthalten	enthalten	enthalten	enthalten	optional
full-platform	enthalten	enthalten	enthalten	enthalten	optional

Abbildung 9: Strukturelle Feature-Zuordnung der fünf Profil-Presets

Tabelle 4: Deklarierte Basisfeatures und resultierende Laufzeitverzeichnisse

Profil	Basisfeatures	Laufzeitverzeichnisse	PostgreSQL
web-only	frontend	frontend/	inkompatibel
web-cloud	frontend, backend, cloud	frontend/, backend/, deployment/	optional
desktop-local	frontend, tauri	frontend/, src-tauri/	inkompatibel
desktop-cloud	frontend, backend, tauri, cloud	frontend/, backend/, src-tauri/, deployment/	optional
full-platform	frontend, backend, tauri, cloud	frontend/, backend/, src-tauri/, deployment/	optional

4.2 Profil Web-only

web-only aktiviert als einziges Plattformfeature frontend. Zum Core kommt daher frontend/. Nicht in den Scaffold gelangen backend/, src-tauri/, deployment/ sowie Datenbank- und PostgreSQL-Abhängigkeiten. Der Generator entfernt außerdem Tauri-Skript und -CLI aus package.json und Lockfile. Das Laufzeitmodell ist damit ein im Browser geladenes Vite-Frontend ohne FastAPI- oder native Desktop-Schicht; auch das generierte Frontend-Profil aktiviert keinen Backend-Statusaufruf. Gegenüber web-cloud fehlen folglich API und Cloud-/Deployment-Feature (kleiveist, 2026d).

4.3 Profil Web-cloud

web-cloud deklariert frontend, backend und cloud. Der Scaffold enthält damit Vite-Frontend, den für das Basisbackend ausgewählten FastAPI-Code sowie deployment/ und .dockerignore; src-tauri/ und dessen npm-Abhängigkeiten werden nicht übernommen. Das Frontend kommuniziert über die öffentliche API-URL mit dem getrennten Backend. cloud kennzeichnet Deployment-Dateien ohne Bindung an einen konkreten Anbieter (kleiveist, 2026b, 2026d).

Ohne weitere Auswahl fehlen Datenbankmodul, Alembic sowie die Datenbank- und Psycpg-Dateien.

PostgreSQL ist deshalb eine kompatible, aber separat anzufordernde Capability; `web-cloud` ist nicht gleichbedeutend mit `web-cloud` + `postgres`.

4.4 Profil Desktop-local

`desktop-local` kombiniert `frontend` und `tauri`; die Abhängigkeit von Tauri zum Frontend ist damit explizit erfüllt. Derselbe Frontend-Build läuft in der Tauri-Webview, während `src-tauri/` die Rust-basierte Desktop-Schicht bereitstellt. Ein FastAPI-Prozess ist hierfür nicht erforderlich: `backend/`, `deployment/` und die Datenbankpfade werden nicht generiert (kleiveist, 2026d).

Das Suffix *local* bezeichnet somit die fehlende Cloud-API. Eine lokale SQL-Datenbank ist nicht eingebaut. Da `database` ein Backend voraussetzt, ist auch die PostgreSQL-Capability mit diesem Profil nicht kompatibel.

4.5 Profil Desktop-cloud

`desktop-cloud` aktiviert `frontend`, `backend`, `tauri` und `cloud`. Es ergänzt `desktop-local` damit um FastAPI und die Deployment-Grenze: Das gemeinsame Frontend wird in der Tauri-Webview ausgeführt und greift über die konfigurierte öffentliche URL auf das getrennte Backend zu. Datenbankinfrastruktur bleibt ohne Capability ausgeschlossen; PostgreSQL kann wegen des vorhandenen Backends ergänzt werden (kleiveist, 2026b, 2026d).

Die Basisfeatures und Feature-Pfade sind identisch zu `full-platform`. Die Profil-ID dokumentiert jedoch den Desktop-Client als vorgesehenen Produktkanal, während das folgende Profil Browser und Desktop gemeinsam adressiert. Ein zusätzlicher technischer Baustein entsteht aus dieser semantischen Abgrenzung nicht.

4.6 Profil Full-platform

Auch `full-platform` deklariert `frontend`, `backend`, `tauri` und `cloud`. Semantisch steht das Preset für zwei Produktkanäle aus derselben Frontend-Basis. Browser-Build und Tauri-Desktopanwendung greifen auf dasselbe FastAPI-Backend zu. Im Scaffold unterscheidet sich das Profil von `desktop-cloud` nur durch ID, Name und Beschreibung im Manifest beziehungsweise im generierten Frontend-Profil; der Scaffold-Plan der Basisfeatures ist gleich (kleiveist, 2026d, 2026g).

Die Bezeichnung *full* schaltet keine Persistenz hinzu. Erst die optionale Auswahl von `postgres` ergänzt `database` und `postgres`; ohne sie bleibt auch dieses Profil datenbankfrei.

4.7 Optionale Capabilities

Der Katalog enthält sechs Features. Vier Abhängigkeiten sind implementiert: `tauri` benötigt `frontend`, `cloud` und `database` benötigen jeweils `backend`, und `postgres` benötigt `database`. Abbildung 10 zeigt nur diese gerichteten Regeln; sie bildet weder Laufzeitkommunikation noch mögliche zukünftige Features ab (kleiveist, 2026a, 2026d).

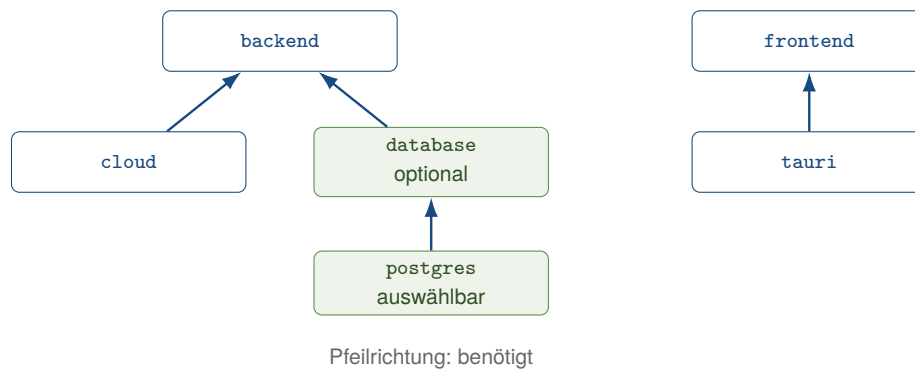


Abbildung 10: Implementierte Abhängigkeiten des Feature-Katalogs

Als benutzerseitig vorgesehene Capability ist gegenwärtig allein `postgres` mit `optional = true` und `selectable = true` markiert. Für ein backendfähiges Profil löst die Auswahl zunächst die ebenfalls optionale, aber nicht als auswählbar markierte Infrastruktur `database` auf. So entsteht beispielsweise `web-cloud + postgres` aus `frontend`, `backend`, `cloud`, `database` und `postgres`. Die Erweiterung fügt SQLAlchemy-/Alembic-Code, Datenbanktests und zugehörige Anforderungsdateien sowie Psycopy-Dateien und PostgreSQL-Integrationstests hinzu. Das gemeinsame Cloud-Compose-Dokument enthält bereits ohne diese Auswahl einen standardmäßigen inaktiven Dienst unter dem Compose-Profil `postgres`; die Capability liefert daher nicht erst dessen Deklaration, sondern die zugehörige Backend-Infrastruktur und Abhängigkeiten (kleiveist, 2026b, 2026g).

Die Auflösung ergänzt ausschließlich fehlende *optionale* Voraussetzungen. Ein nicht optionales Plattformfeature wird nicht stillschweigend aktiviert. Deshalb lehnt das Tooling `web-only + postgres` und `desktop-local + postgres` ab: beiden fehlt `backend`. Neue Capabilities folgen demselben dokumentierten Weg über Katalogeintrag, Abhängigkeiten, eigene Pfade und Tests; als implementiert gelten sie dadurch noch nicht.

Der Vergleich von Dokumentation und Code zeigt zwei Abweichungen. Die interaktive Auswahl bietet entsprechend `selectable` nur PostgreSQL an; bei einer direkten `-with database`-Angabe prüft der Resolver jedoch nur `optional` und akzeptiert die generische Datenbankfähigkeit für Backendprofile. Ferner liegen die Einträge für die generierte `.env.example` nicht in `features.toml`, wie die Profildokumentation allgemein formuliert, sondern werden anhand von `config/environment.toml` und den aufgelösten Features ausgewählt. Diese Unterschiede ändern die fünf Presets nicht, begrenzen aber die deklarierte Trennung zwischen interner und auswählbarer Capability.

4.8 Single-Repository-Strategie

Die fünf Profile sind keine langfristig gepflegten Template-Forks. Im Master-Repository liegen Core, vollständige Feature-Implementierungen, Profildefinitionen und Generator gemeinsam; `features.toml` und die fünf Preset-Dateien bilden dabei die deklarative Quelle für Pfade und Abhängigkeiten. Damit existiert eine technische Wartungsbasis, aus der der Generator eigenständige Produktprojekte ableitet (kleiveist, 2026d, 2026g).

Master-Repository und Produktprojekt sind folglich zu unterscheiden: Das Master enthält auch optionale Implementierungen und aktiviert in seinem eigenen Manifest PostgreSQL, damit diese Basis wartbar bleibt. Das generierte Projekt erhält Core plus die Pfade seines aufgelösten Profils, ein neu-

es `project-profile.toml`, das generierte Frontend-Profil und eine featureabhängige `.env.example`. Es bleibt anschließend ein separates Projektverzeichnis; der Generator verändert das Master nicht. Die Struktur reduziert konzeptionell die Zahl unabhängiger Template-Baselines und adressiert damit die in Kapitel 2 formulierte kontrollierte Variabilität. Eine vergleichende Bewertung dieser Konsequenz erfolgt erst im Bewertungskapitel.

5 Tooling und Entwicklungsworkflow

5.1 Rolle von `control.py`

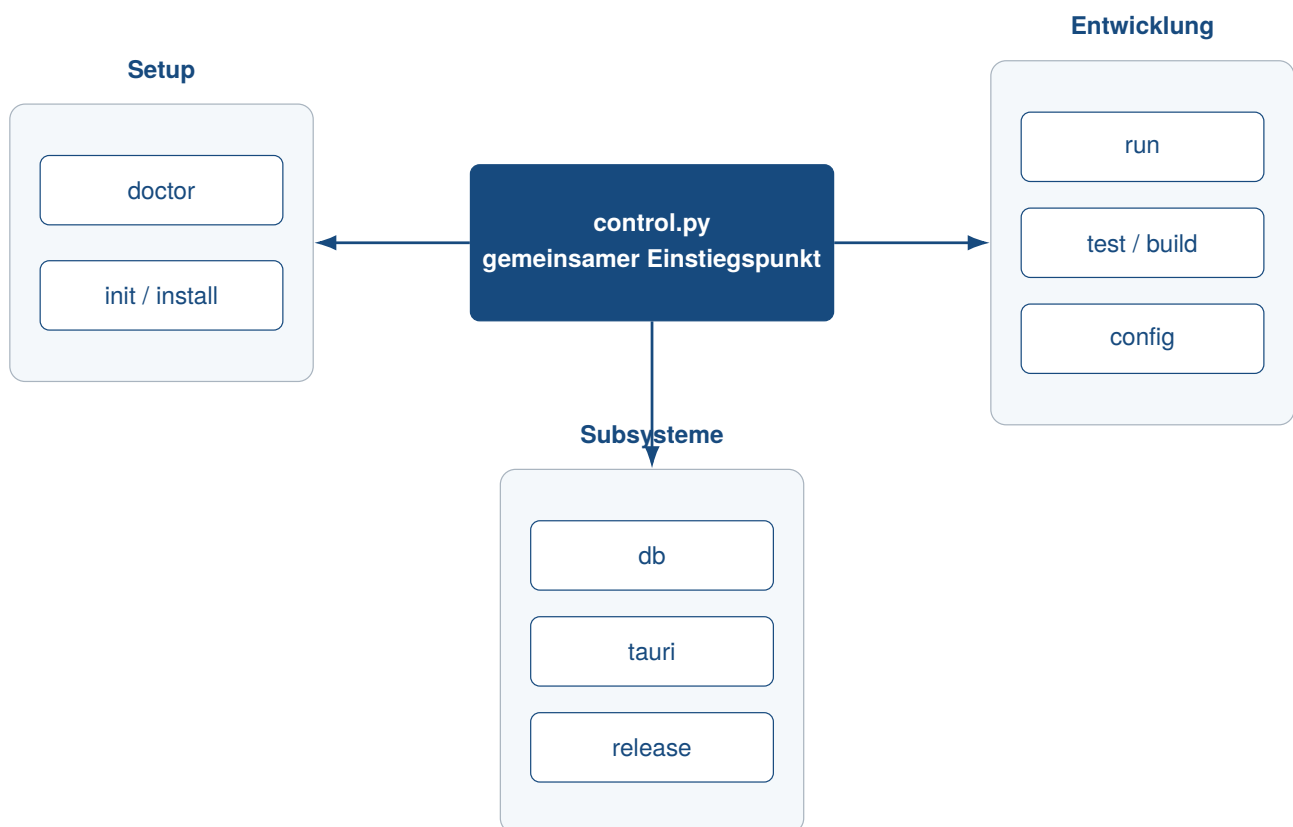


Abbildung 11: Befehlsgruppen des zentralen Projekt-Toolings

5.2 Projektinitialisierung und Generierung

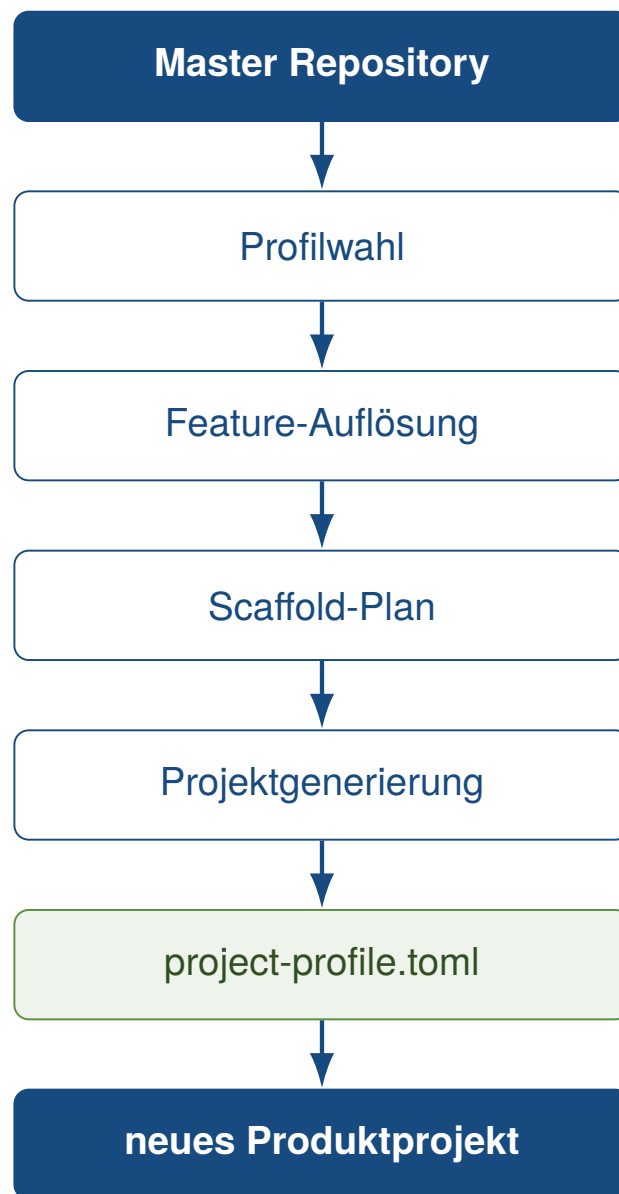


Abbildung 12: Workflow der profilgesteuerten Projektgenerierung

5.3 Systemprüfung und Installation

5.4 Entwicklungsbetrieb



Abbildung 13: Grundstruktur des Entwicklungsworkflows

5.5 Tests und Builds

5.6 Release- und Packaging-Workflow

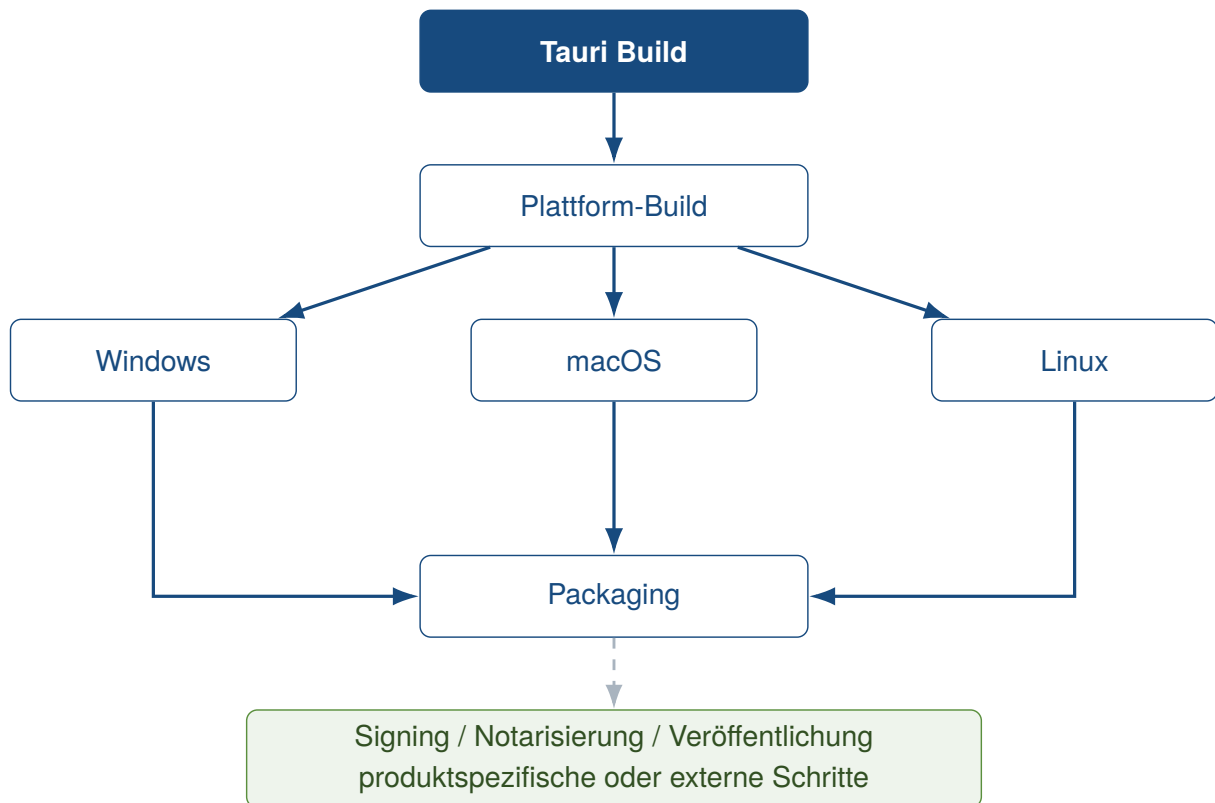


Abbildung 14: Plattformübergreifendes Modell für Desktop-Releases

5.7 Entwickler-Onboarding

6 Qualitätssicherung und Verifikation

6.1 Teststrategie



Abbildung 15: Nachweiskette der technischen Verifikation

6.2 Continuous Integration

6.3 Abnahme und technische Verifikation

Profil	Generate	Install	Doctor	Tests	Web Build	Desktop / Tauri	Database	Container
web-only	offen	offen	offen	offen	offen	offen	offen	offen
web-cloud	offen	offen	offen	offen	offen	offen	offen	offen
desktop-local	offen	offen	offen	offen	offen	offen	offen	offen
desktop-cloud	offen	offen	offen	offen	offen	offen	offen	offen
full-platform	offen	offen	offen	offen	offen	offen	offen	offen

Abbildung 16: Struktur der profilbezogenen Verifikationsmatrix

Tabelle 5: Struktur zur Dokumentation der technischen Verifikation

Profil	Prüfbereich	Kriterium	Evidenz	Ergebnis
--------	-------------	-----------	---------	----------

6.4 Reproduzierbarkeit

6.5 Security und Konfigurationsgrenzen

6.6 Extern verbleibende Prüfungen

7 Bewertung des Technologie-Templates

7.1 Produktivitätswirkung

7.2 Stärken

Tabelle 6: Struktur zur Gegenüberstellung von Stärken und Grenzen

Aspekt	Stärke / Evidenz	Grenze / Evidenz	Auswirkung
--------	------------------	------------------	------------

7.3 Schwächen und Grenzen

7.4 Wartungsaufwand

7.5 Vergleich mit alternativen Template-Strategien

7.6 Gesamtbewertung

8 Einsatzmöglichkeiten und Transfer auf Kundenprojekte

8.1 Geeignete Projekttypen

Projekttyp	Anforderungsfit	Anpassungsbedarf	Evidenz
Web-Anwendungen	offen	offen	offen
Cloud-Anwendungen	offen	offen	offen
Desktop-Anwendungen	offen	offen	offen
SaaS + Desktop	offen	offen	offen
lokale Business-Tools	offen	offen	offen
stark native Anwendungen	offen	offen	offen
Echtzeitsysteme	offen	offen	offen
Spiele	offen	offen	offen

Abbildung 17: Offene Bewertungsstruktur für die Projekteignung

Tabelle 7: Struktur zur Bewertung der Projekteignung

Projekttyp	Anforderungsfit	Anpassungsbedarf	Grenzen	Evidenz
------------	-----------------	------------------	---------	---------

8.2 Ungeeignete und eingeschränkt geeignete Projekttypen

8.3 Analyse von Kundenanforderungen

8.4 Auswahl des Projektprofils

8.5 Generierung eines Kundenprojekts



Abbildung 18: Prozess vom Kundenbedarf bis zur Auslieferung

8.6 Überführung in die Produktentwicklung

9 Schlussbetrachtung

9.1 Zusammenfassung der Ergebnisse

9.2 Beantwortung der Fragestellungen

9.3 Kritische Würdigung

9.4 Ausblick

9.5 Fazit

Literaturverzeichnis

- Bayer, M. (2026). *Alembic tutorial* [Alembic 1.19.1 Documentation]. Verfügbar 8. August 2026 unter <https://alembic.sqlalchemy.org/en/latest/tutorial.html>
- Clements, P. C., & Northrop, L. M. (2001). *Software product lines: Practices and patterns*. Addison-Wesley Professional. Verfügbar 8. August 2026 unter <https://www.sei.cmu.edu/library/software-product-lines-practices-and-patterns/>
- ISO/IEC. (2023). *Systems and software engineering—systems and software quality requirements and evaluation (square)—product quality model* (International Standard ISO/IEC 25010:2023). International Organization for Standardization. Verfügbar 8. August 2026 unter <https://www.iso.org/standard/78176.html>
- Klabnik, S., Nichols, C., & Krycho, C. (2026). *The rust programming language* [Official Rust Documentation]. Verfügbar 8. August 2026 unter <https://doc.rust-lang.org/book/>
- kleiveist. (2026a, 6. August). *Database feature* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db89620docs/def/database-feature.md>
- kleiveist. (2026b, 6. August). *Deployment architecture* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db8962009ff6c13567c319dfdb/docs/def/deployment-architecture.md>
- kleiveist. (2026c, 6. August). *Framework architecture* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db8962009ff6c13567c319dfdb/docs/def/architecture.md>
- kleiveist. (2026d, 6. August). *Project profiles* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db89620docs/def/project-profiles.md>
- kleiveist. (2026e, 6. August). *Runtime configuration* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db8962009ff6c13567c319dfdb/docs/def/configuration.md>
- kleiveist. (2026f, 7. August). *Template final acceptance* [Technisches Abnahmeprotokoll im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db8962009ff6c13567c319dfdb/docs/dev/template-final-acceptance.md>
- kleiveist. (2026g, 7. August). *Template-projekte: Full-stack project template* [GitHub-Repository, geprüfter Commit a4487ac]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte>
- kleiveist. (2026h, 6. August). *Tooling guide* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db89620docs/tools/tooling.md>
- Krueger, C. W. (1992). Software reuse. *ACM Computing Surveys*, 24(2), 131–183. <https://doi.org/10.1145/130844.130856>
- Lamb, C., & Zacchiroli, S. (2022). Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2), 62–70. <https://doi.org/10.1109/MS.2021.3073045>
- Microsoft. (2026, 27. Juli). *The typescript handbook* [TypeScript Documentation]. Verfügbar 8. August 2026 unter <https://www.typescriptlang.org/docs/handbook/intro.html>

- PostgreSQL Global Development Group. (2026). *Postgresql 18.4 documentation*. Verfügbar 8. August 2026 unter <https://www.postgresql.org/docs/current/index.html>
- Ramírez, S. (2026). *Fastapi features* [Official FastAPI Documentation]. Verfügbar 8. August 2026 unter <https://fastapi.tiangolo.com/features/>
- Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>
- SQLAlchemy Authors and Contributors. (2026, 15. Juni). *Engine configuration* [SQLAlchemy 2.0 Documentation, Release 2.0.51]. Verfügbar 8. August 2026 unter <https://docs.sqlalchemy.org/en/20/core/engines.html>
- Tauri Contributors. (2026a). *Capabilities* [Tauri 2 Documentation]. Verfügbar 8. August 2026 unter <https://v2.tauri.app/security/capabilities/>
- Tauri Contributors. (2026b, 22. Juli). *What is tauri?* [Tauri 2 Documentation]. Verfügbar 8. August 2026 unter <https://v2.tauri.app/start/>
- VoidZero Inc. and Vite Contributors. (2026). *Getting started* [Vite Documentation]. Verfügbar 8. August 2026 unter <https://vite.dev/guide/>